UNIVERSIDADE PAULISTA – UNIP
CIÊNCIA DA COMPUTAÇÃO

Vinicius Oliveira dos Santos
Gabriel Boaron Dalsas Resende
Carlos Eduardo Martins Penna
Felipe Cantareli Andrucioli
Davi Fernando Sousa Dias

ECOPULSE:
Plataforma distribuída mobile para monitoramento colaborativo de riscos ambientais urbanos e qualidade do ar

Ribeirão Preto
2026

Vinicius Oliveira dos Santos
Gabriel Boaron Dalsas Resende
Carlos Eduardo Martins Penna
Felipe Cantareli Andrucioli
Davi Fernando Sousa Dias

ECOPULSE:
Plataforma distribuída mobile para monitoramento colaborativo de riscos ambientais urbanos e qualidade do ar

Atividade Prática Supervisionada (APS) apresentada à disciplina Desenvolvimento de Sistemas Distribuídos (DSD) do Curso de Ciência da Computação da Universidade Paulista – UNIP, 8º semestre, 2026/2.

Professor: Jean Scatena

Ribeirão Preto
2026

RESUMO

[Parágrafo único, sem recuo, de 150 a 500 palavras: contexto do problema, objetivo do trabalho, método (arquitetura orientada a eventos e microsserviços), resultados obtidos e considerações finais.]

Palavras-chave: [palavra 1]. [palavra 2]. [palavra 3].

LISTA DE FIGURAS

Figura 1 – Arquitetura do EcoPulse 30

LISTA DE QUADROS

Quadro 1 – Divisão de responsabilidades da equipe 20
Quadro 2 – Requisitos do EcoPulse 21
Quadro 3 – Casos de uso do EcoPulse 24
Quadro 4 – Cronograma e etapas do EcoPulse 26
Quadro 5 – Casos de teste do EcoPulse 27
Quadro 6 – Riscos e mitigação do EcoPulse 29
Quadro 7 – Componentes e responsabilidades do EcoPulse 31
Quadro 8 – Dados do EcoPulse 33
Quadro 9 – Comportamento do EcoPulse diante de falhas 33

LISTA DE ABREVIATURAS E SIGLAS

API – Application Programming Interface (Interface de Programação de Aplicações)

APS – Atividade Prática Supervisionada

AQI – Air Quality Index (Índice de Qualidade do Ar)

CAP – Consistency, Availability, Partition tolerance (Consistência, Disponibilidade, Tolerância a Partições)

CEMADEN – Centro Nacional de Monitoramento e Alertas de Desastres Naturais

DLQ – Dead Letter Queue (Fila de Mensagens Mortas)

DSD – Desenvolvimento de Sistemas Distribuídos

GiST – Generalized Search Tree (Árvore de Busca Generalizada)

GPS – Global Positioning System (Sistema de Posicionamento Global)

HTTP – Hypertext Transfer Protocol (Protocolo de Transferência de Hipertexto)

IPCC – Intergovernmental Panel on Climate Change (Painel Intergovernamental sobre Mudanças Climáticas)

RF – Requisito Funcional

RFC – Request for Comments

RNF – Requisito Não Funcional

SQL – Structured Query Language (Linguagem de Consulta Estruturada)

SQS – Simple Queue Service

UNIP – Universidade Paulista

YAML – YAML Ain't Markup Language

SUMÁRIO

1 OBJETIVO E MOTIVAÇÃO 8
1.1 Objetivo geral 8
1.2 Objetivos específicos 8
1.3 Motivação 9
2 INTRODUÇÃO 10
3 FUNDAMENTOS DAS TECNOLOGIAS ESCOLHIDAS 13
3.1 Sistemas distribuídos, tolerância a falhas e consistência eventual 13
3.2 Arquitetura orientada a eventos e microsserviços 13
3.3 Mensageria assíncrona com RabbitMQ 14
3.4 Dados geoespaciais: PostgreSQL com PostGIS e índice H3 15
3.5 Cache e publicação/assinatura com Redis 15
3.6 Comunicação em tempo real com WebSocket 16
3.7 Aplicativo móvel: React Native (Expo), MapLibre e SQLite 17
3.8 Node.js e contêineres com Docker 17
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO 19
4.1 Metodologia de desenvolvimento 19
4.2 Requisitos 21
4.3 Casos de uso 23
4.4 Cronograma e etapas 25
4.5 Estratégia de testes 27
4.6 Riscos e mitigação 28
5 PROJETO E ARQUITETURA DO PROGRAMA 30
5.1 Visão geral da arquitetura 30
5.2 Microsserviços e responsabilidades 31
5.3 Fluxo de eventos e mensageria 31
5.4 Modelo de dados 32
5.5 Tolerância a falhas e escalabilidade 33
6 RELATÓRIO DE CÓDIGO 35
6.1 Estrutura do repositório 36
6.2 Trechos relevantes do código 36
7 APRESENTAÇÃO EM FUNCIONAMENTO 37
7.1 Ambiente de execução 37
7.2 Demonstração das funcionalidades 37
7.3 Resultados dos testes 37
8 CONSIDERAÇÕES FINAIS 38
REFERÊNCIAS 39
ANEXO A – Ficha de APS 41

1 OBJETIVO E MOTIVAÇÃO

1.1 Objetivo geral

Desenvolver o EcoPulse, uma aplicação móvel integrada a um sistema distribuído orientado a eventos, capaz de exibir em um mapa o calor, a qualidade do ar, os alagamentos e as vias obstruídas de uma região, de receber relatos colaborativos dos usuários e de alertá-los em tempo real quando se aproximarem de um risco ambiental, aplicando na prática os principais conceitos da disciplina de Desenvolvimento de Sistemas Distribuídos.

1.2 Objetivos específicos

Projetar uma arquitetura orientada a eventos em que a API apenas receba e publique os pedidos, e processos independentes (workers) os concluam, desacoplados por mensageria assíncrona com RabbitMQ;

Garantir tolerância a falhas na ingestão de relatos por meio de filas duráveis, confirmação manual de processamento, novas tentativas com espera crescente e fila de mensagens mortas;

Particionar os dados geograficamente com índices H3 e armazená-los em PostgreSQL com PostGIS, de modo que cada consulta trabalhe apenas a região visível no mapa;

Utilizar o Redis como cache de leitura e como canal de publicação e assinatura (pub/sub), aceitando consistência eventual entre o cache e o banco de dados;

Entregar alertas de proximidade em tempo real, por WebSocket, aos clientes conectados;

Desenvolver um aplicativo em React Native (Expo), com mapa MapLibre, relato com localização automática pelo GPS e armazenamento local em SQLite, para que relatos feitos sem conexão sejam enviados quando a rede voltar;

Integrar a API pública Open-Meteo para obter temperatura e qualidade do ar, mantendo as leituras em cache para limitar as consultas externas.

1.3 Motivação

As cidades enfrentam, de forma recorrente, alagamentos repentinos, ilhas de calor e episódios de poluição do ar, agravados pelo crescimento desordenado e por eventos climáticos extremos. Nesse cenário, a informação só é útil se chegar a tempo e com precisão local. As abordagens centralizadas tradicionais costumam ter baixa resolução espacial e temporal, o que limita a capacidade de avisar a população quando o risco ainda pode ser evitado.

Há, porém, uma diferença importante entre os tipos de risco. Temperatura e qualidade do ar já são medidas e publicadas por serviços meteorológicos, como a Open-Meteo, e podem ser consultadas para qualquer região. Já o alagamento e a obstrução de uma via dependem de quem está no local: até onde se pôde apurar, não há um serviço aberto e de cobertura mundial que informe, em tempo real, que determinada rua está alagada ou fechada. É nesse ponto que o celular se torna útil, pois pode funcionar ao mesmo tempo como sensor, quando o usuário relata o que viu, e como receptor, quando recebe o alerta de um risco próximo.

Esse modelo colaborativo traz problemas típicos de sistemas distribuídos. Em uma chuva forte, por exemplo, muitos usuários podem relatar ao mesmo tempo, justamente quando a infraestrutura está mais pressionada. O sistema precisa absorver esses picos sem perder relatos, continuar respondendo quando uma de suas partes falha, atualizar o mapa dos usuários em tempo real e aceitar que cache e banco de dados fiquem momentaneamente diferentes. Esses requisitos justificam o uso de mensageria assíncrona, filas com novas tentativas, particionamento geográfico, cache distribuído e comunicação em tempo real, conteúdos centrais da disciplina.

Por fim, o projeto reúne esses conceitos em uma aplicação completa e verificável. O mapa é mundial, mas a demonstração acontece em Ribeirão Preto, com dois clientes conectados à mesma API: um deles publica um relato e o outro recebe o alerta. Assim, cada decisão de arquitetura descrita neste relatório pode ser observada em funcionamento.

2 INTRODUÇÃO

O crescimento das cidades, muitas vezes de forma desordenada, aumenta a exposição da população a riscos ambientais que as mudanças climáticas tendem a agravar. O Painel Intergovernamental sobre Mudanças Climáticas (IPCC) avalia que as ondas de calor devem se tornar mais perigosas nas cidades, em parte pela ilha de calor urbana, e que chuvas extremas mais frequentes e intensas ampliam a área urbana exposta a inundações (Dodman et al., 2022).

No Brasil, o quadro já é concreto. Um levantamento de quase 60 mil registros de desastres relacionados à água entre 1991 e 2024 mostra que 91,5% dos municípios brasileiros registraram ao menos um evento desse tipo (Escobar-Silva et al., 2026). Os órgãos oficiais, como o Centro Nacional de Monitoramento e Alertas de Desastres Naturais (CEMADEN), respondem com modelos de previsão e sensores de superfície (CEMADEN, 2022). Esse monitoramento é indispensável, mas pode não alcançar, sozinho, o que acontece em uma rua específica neste momento. Goodchild (2007) observa que as pessoas guardam um conhecimento local de difícil captação por sensores remotos e propõe a informação geográfica voluntária como fonte complementar, na qual cada pessoa atua como um sensor.

O telefone celular é a plataforma natural para esse papel, pois reúne GPS, conexão de dados e uma tela por onde chega o aviso. O mesmo aparelho que registra uma ocorrência pode receber o alerta de um risco próximo: o usuário é, ao mesmo tempo, sensor e destinatário da informação.

Construir um serviço assim, porém, exige mais do que um aplicativo e um banco de dados. Tanenbaum e Van Steen (2007) definem um sistema distribuído como um conjunto de computadores independentes que se apresenta aos usuários como um sistema único e coerente. Em um sistema de relatos colaborativos, as questões desse tipo de sistema aparecem na prática: os relatos chegam em rajadas, partes do sistema podem falhar sem que nenhum relato seja perdido, o mapa de cada usuário precisa ser atualizado em tempo real e cada consulta deve trabalhar apenas a região que o usuário está vendo.

A informação colaborativa traz, porém, uma dificuldade própria: ela não passa pelo controle de qualidade de uma rede oficial de sensores. Um único relato pode estar equivocado, ser antigo ou ser mal-intencionado, e um alagamento que já secou não deve continuar no mapa. Um serviço desse tipo precisa, portanto, decidir

quando acreditar em um relato e por quanto tempo mantê-lo, além de limitar o volume que cada aparelho pode enviar. Essas decisões também fazem parte do projeto e são tratadas nas seções seguintes.

Diante desse cenário, a questão que orienta este trabalho é como projetar um sistema distribuído que receba relatos de muitos usuários, mantenha os dados geográficos organizados, continue funcionando quando uma de suas partes falhar e entregue o alerta a quem está perto do risco no momento em que ele surge. A resposta proposta é um protótipo funcional, descrito a seguir.

Este trabalho propõe o EcoPulse, um aplicativo móvel de mapa mundial que exibe quatro camadas de risco ambiental: calor, qualidade do ar, alagamento e via obstruída. O usuário liga e desliga cada camada, toca em uma região para ver os valores, envia relatos com a localização preenchida automaticamente pelo GPS e recebe uma faixa de alerta ao se aproximar de um risco. Sem rede, o relato fica guardado no aparelho e é enviado quando a conexão volta. Calor e qualidade do ar vêm da API pública Open-Meteo (Open-Meteo, [s. d.]). Alagamento e via obstruída, para os quais essa fonte não existe, vêm exclusivamente dos relatos dos usuários.

Para isso, o EcoPulse adota uma arquitetura orientada a eventos. O aplicativo, desenvolvido em React Native, não grava o incidente diretamente no banco de dados: ele entrega o relato a uma API em Node.js, que o publica em uma fila do RabbitMQ e responde de imediato. Processos independentes consomem a fila, gravam os dados em um banco PostgreSQL com PostGIS, atualizam o cache em Redis e geram os alertas, que chegam aos clientes por WebSocket. O espaço é dividido em células hexagonais H3, de modo que cada consulta trabalhe apenas um pedaço do mapa. As escolhas tecnológicas e seus motivos são detalhados na seção 3.

O protótipo é executado em ambiente local, com os serviços em containers Docker, e foi pensado para ser demonstrado com dois clientes conectados à mesma API. Ficam fora desta versão a implantação em produção, a notificação push com o aplicativo fechado e o traçado de rotas alternativas, registrados neste relatório como evoluções possíveis.

Além desta introdução, o relatório está organizado da seguinte forma. A seção 1 apresenta os objetivos e a motivação. A seção 3 reúne os fundamentos das tecnologias escolhidas, a seção 4 descreve o plano de desenvolvimento e a seção 5 detalha o projeto e a arquitetura do programa. A seção 6 traz o relatório de código, a

seção 7 mostra a aplicação em funcionamento e a seção 8 apresenta as considerações finais, seguidas das referências.

3 FUNDAMENTOS DAS TECNOLOGIAS ESCOLHIDAS

Esta seção apresenta os conceitos e as tecnologias que sustentam o EcoPulse. Cada subseção explica o que a tecnologia é, por que foi escolhida e qual papel ela cumpre no sistema, indo dos conceitos gerais de sistemas distribuídos até as ferramentas usadas no aplicativo e na infraestrutura.

3.1 Sistemas distribuídos, tolerância a falhas e consistência eventual

Em um sistema distribuído, os componentes ficam em processos ou máquinas diferentes e se comunicam por uma rede, o que torna as falhas parciais uma situação normal, e não uma exceção. Para Kleppmann (2017), um sistema confiável continua funcionando corretamente mesmo quando algo dá errado, e essa tolerância a falhas precisa ser equilibrada com outras propriedades, como o desempenho e a consistência dos dados.

Essa relação aparece no teorema CAP, segundo o qual um sistema de armazenamento distribuído que privilegia a consistência abre mão de parte da disponibilidade, e o que privilegia a disponibilidade precisa aceitar uma consistência mais fraca. Ferreira, Andrade e Mendonça (2021) descrevem a consistência eventual como um modelo em que as atualizações são persistidas de forma assíncrona, de modo que réplicas podem ficar desatualizadas por algum tempo até convergirem, e mostram que níveis mais fortes de consistência aumentam o tempo de resposta.

No EcoPulse, esse equilíbrio é uma decisão de projeto. O PostgreSQL é o registro definitivo dos relatos, e o cache em Redis atende à leitura rápida do mapa. Um relato é aceito pela fila, depois gravado no banco e só então refletido no cache, de modo que, por um instante, o mapa de um usuário pode mostrar um dado um pouco mais antigo que o do banco. Aceitar essa defasagem breve permite ao sistema responder rápido e continuar funcionando quando uma de suas partes está lenta ou fora do ar.

3.2 Arquitetura orientada a eventos e microsserviços

Em uma arquitetura orientada a eventos, os componentes não se chamam diretamente: um componente registra um fato, o evento, e os demais reagem a ele

quando puderem. Esse modelo costuma ser combinado com microsserviços, em que a aplicação é dividida em pequenos serviços, cada um em seu próprio processo. Daniel et al. (2020) observam que essa divisão melhora a escalabilidade e a manutenção, mas cria concessões entre a consistência dos dados e o acoplamento entre serviços, e apresentam um arcabouço de microsserviços orientados a eventos como resposta.

No EcoPulse, há quatro processos de aplicação. A API recebe as requisições do aplicativo, faz uma validação mínima, publica o evento e responde sem esperar o processamento. O worker de relatos consome os relatos, grava os dados e gera os alertas, e o worker de clima consulta a Open-Meteo para as regiões visíveis no mapa. O quarto processo é o próprio aplicativo móvel. Como a API não executa o trabalho pesado, um pico de relatos faz a fila crescer em vez de derrubar o serviço.

3.3 Mensageria assíncrona com RabbitMQ

O RabbitMQ é um message broker: um intermediário que recebe mensagens de quem as publica e as entrega a quem as consome, por meio de filas. Oliveira e Fazenda (2026) compararam o RabbitMQ ao Amazon Simple Queue Service (SQS) e relatam que o RabbitMQ teve menores latências e maiores taxas de processamento, enquanto o SQS se destaca pelas vantagens de um serviço gerenciado na nuvem. No EcoPulse, que roda localmente em contêineres, o RabbitMQ atende ao desacoplamento, às novas tentativas e à fila de mensagens mortas com um único serviço.

Os recursos de confiabilidade do RabbitMQ são centrais para o projeto. No modo de confirmação manual, o broker só considera uma mensagem tratada depois que o consumidor envia uma confirmação explícita (ack), e, se o consumidor falhar antes disso, a mensagem volta para a fila. A documentação lembra que uma mensagem pode ser entregue mais de uma vez, de modo que os consumidores devem ser idempotentes (RabbitMQ). Mensagens que não podem ser processadas podem ser reencaminhadas a uma dead letter exchange, base da fila de mensagens mortas (DLQ), em que o EcoPulse guarda o que não conseguiu processar para análise posterior.

No projeto, as filas são duráveis, as mensagens são persistentes e o relato só é confirmado depois de gravado no PostgreSQL. Uma mensagem com formato inválido vai direto para a DLQ, porque repetir o processamento não corrige um dado

quebrado. Já uma falha passageira, como um banco de dados fora do ar, provoca até três novas tentativas, com espera crescente, antes de a mensagem seguir para a DLQ. Assim, nenhum relato é descartado em silêncio.

3.4 Dados geoespaciais: PostgreSQL com PostGIS e índice H3

O PostGIS é uma extensão do sistema de banco de dados objeto-relacional PostgreSQL que permite armazenar objetos geográficos no banco e inclui suporte a índices espaciais baseados em GiST R-Tree (PostGIS Development Group, 2026). A importância dos índices aparece no trabalho de Santos et al. (2017), em que o uso de índices e o pré-cálculo de uma interseção espacial reduziram o tempo de uma consulta em PostGIS de cerca de 7 minutos para cerca de 5 segundos. No EcoPulse, cada relato é guardado como um ponto geográfico e o mapa é lido sempre pelo retângulo visível, nunca pelo planeta inteiro. O índice espacial permite buscar apenas os pontos dentro desse retângulo ou perto de uma posição, que é a consulta típica de um alerta de proximidade.

Para dividir o espaço em pedaços de tamanho previsível, o sistema usa o H3, um índice espacial hexagonal hierárquico criado pela Uber. Brodsky (2018) explica que o H3 divide a superfície da Terra em hexágonos, em 16 resoluções nas quais cada nível mais fino tem cerca de um sétimo da área do anterior, e que o identificador de uma célula pode ser truncado para encontrar a célula-mãe em uma resolução mais grossa.

No EcoPulse, o H3 aparece em duas resoluções. Uma resolução mais grossa define as células de calor e de qualidade do ar, de modo que cada pedido à Open-Meteo e cada entrada de cache corresponde a uma célula. Uma resolução mais fina agrupa relatos próximos para formar um incidente, e dois relatos do mesmo tipo na mesma célula confirmam o incidente. Como cada consulta trabalha apenas algumas células, o desenho favorece a divisão dos dados e da carga por região, que é a ideia de particionamento geoespacial da disciplina.

3.5 Cache e publicação/assinatura com Redis

O Redis é um armazenamento de dados em memória, usado no projeto como cache e como canal de mensagens. Tenório (2022) propõe o Redis como cache distribuído para dados muito solicitados nos sistemas do Tribunal de Contas do

Estado do Rio Grande do Norte e, em testes de carga, relata melhora no tempo de resposta e menor carga sobre os bancos de dados. Cada chave pode receber um prazo de validade e, passado esse prazo, é apagada automaticamente, o que combina com dados que perdem o valor com o tempo, como uma leitura de temperatura.

O Redis também oferece publicação e assinatura (pub/sub): publicadores enviam mensagens a canais, e todos os clientes inscritos no canal as recebem, sem que uns conheçam os outros. A documentação esclarece que a entrega é no máximo uma vez: se o assinante não conseguir tratar a mensagem, ela é perdida (Redis). Por isso, o PostgreSQL continua sendo o registro dos relatos.

No EcoPulse, as leituras de calor e de qualidade do ar ficam no cache por 10 minutos, e os incidentes ativos por 6 horas, renovadas a cada novo relato. O pub/sub leva os eventos de camada atualizada e de alerta gerado até a API, que os repassa aos clientes. Perder o Redis deixa a leitura seguinte mais lenta, mas não perde relatos. Na versão local, o Redis roda em um único contêiner. O relatório trata o papel de cache distribuído, e a distribuição em vários nós fica como evolução.

3.6 Comunicação em tempo real com WebSocket

No modelo tradicional do HTTP, é o cliente que precisa perguntar para receber algo. O WebSocket, definido na RFC 6455 (Fette; Melnikov, 2011), permite a comunicação nos dois sentidos entre um cliente e um servidor por meio de uma conexão que permanece aberta, de modo que o servidor pode enviar dados assim que houver novidade.

No EcoPulse, o aplicativo abre uma conexão WebSocket com a API e informa o retângulo visível do mapa e, se houver GPS, a posição do usuário. Quando o Redis publica um evento de camada atualizada ou de alerta gerado, a API o envia apenas aos clientes cujo retângulo ou posição é afetado. É assim que um relato feito em um celular pode aparecer em outro sem que o usuário atualize a tela. O servidor usa a biblioteca ws, para Node.js. Esse mecanismo funciona com o aplicativo aberto ou em segundo plano imediato, e a notificação push com o aplicativo fechado fica como evolução.

3.7 Aplicativo móvel: React Native (Expo), MapLibre e SQLite

O aplicativo é desenvolvido em React Native, um framework para construir aplicativos para Android, iOS e web usando JavaScript e React. Com o Expo, o mesmo código roda no celular físico e no navegador, o que permite demonstrar dois clientes ao mesmo tempo. A escolha troca um pouco do desempenho de um aplicativo totalmente nativo por uma única base de código, adequada ao prazo e ao foco da disciplina, que está no sistema distribuído por trás do aplicativo.

Para o mapa, o EcoPulse usa o MapLibre, uma biblioteca de código aberto para mapas, com versões para a web e para aplicativos móveis. O projeto o prefere a um serviço de mapas comercial para não depender de chave de acesso nem de cobrança por uso, o que importa em um mapa mundial que o usuário pode arrastar à vontade.

A tolerância a falhas de rede no aparelho depende do armazenamento local. O SQLite é um motor de banco de dados SQL embutido no aplicativo, sem processo servidor separado, e a biblioteca expo-sqlite dá acesso a ele no Expo. Essa é a base da abordagem offline-first, em que o aplicativo trabalha primeiro com dados locais e sincroniza depois. Confessor et al. (2025) propõem um aplicativo para coleta de dados de pesquisa em áreas com conectividade limitada, que grava os dados em SQLite e os sincroniza automaticamente quando a conexão volta. O EcoPulse segue a mesma lógica, mas sincroniza com a própria API: relatos feitos sem rede ficam em uma fila local, com estados como pendente, enviado e falhado, e são enviados em ordem quando a rede volta. A última camada vista também fica guardada, para consulta sem conexão.

3.8 Node.js e contêineres com Docker

O backend é escrito em Node.js, um ambiente de execução JavaScript assíncrono e orientado a eventos. Como nada bloqueia enquanto se espera uma resposta de fila, banco de dados ou rede, o modelo combina com serviços que passam a maior parte do tempo esperando, como a API e os workers do EcoPulse. A API usa o framework Express.

Os serviços rodam em contêineres Docker. Trindade (2018) descreve o contêiner como semelhante a uma máquina virtual, mas possivelmente mais eficiente, porque os contêineres compartilham o kernel do sistema hospedeiro e

isolam processos no nível do sistema operacional, o que evita a sobrecarga do hardware virtualizado. O Docker Compose permite definir todos os serviços em um único arquivo YAML e iniciá-los com um único comando. No EcoPulse, o Compose sobe o PostgreSQL com PostGIS, o Redis, o RabbitMQ, a API e os dois workers, o que torna o ambiente reproduzível em qualquer máquina do grupo. O aplicativo Expo roda fora do Compose, no celular ou no navegador, apontando para o endereço da API na rede local.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Esta seção descreve como o EcoPulse foi planejado e construído. Ela apresenta a metodologia adotada pela equipe, os requisitos do sistema e seus casos de uso, o cronograma das etapas, a estratégia de testes e os riscos identificados, com as respectivas medidas de mitigação. O objetivo é registrar o processo que levou do problema descrito nas seções anteriores ao protótipo funcional detalhado nas seções seguintes.

4.1 Metodologia de desenvolvimento

O desenvolvimento do EcoPulse seguiu um processo iterativo e incremental, com práticas inspiradas no Scrum e adaptadas à realidade de um grupo de cinco alunos ao longo de um semestre. O trabalho foi dividido em funcionalidades (features), cada uma com prazo de entrega semanal, de modo que cada ciclo, equivalente a uma sprint, durou uma semana. A cada ciclo, uma parte do sistema era concluída e integrada ao que já existia, o que permitiu ao produto crescer de forma gradual e tornou possível identificar problemas cedo. Não foram adotados todos os papéis e cerimônias do Scrum completo, como Product Owner, Scrum Master, reuniões diárias e retrospectivas formais. Mantiveram-se o backlog, as entregas curtas e o foco em incrementos funcionais.

A comunicação da equipe foi feita pelo WhatsApp e pelo Discord. O alinhamento foi contínuo, por mensagens, conforme a necessidade, como na conclusão de uma etapa ou em uma decisão que afetava mais de uma parte do sistema.

A divisão de tarefas considerou a área de maior conhecimento de cada integrante, conforme o Quadro 1.

Quadro 1 – Divisão de responsabilidades da equipe

Integrante	Responsabilidade
Felipe Cantareli Andrucioli	Infraestrutura (contêineres e serviços de apoio)
Davi Fernando Sousa Dias	Desenvolvimento do aplicativo móvel (React Native com Expo)
Vinicius Oliveira dos Santos e Carlos Eduardo Martins Penna	Desenvolvimento da API
Gabriel Boaron Dalsas Resende	Documentação do projeto

Fonte: Elaborado pelos autores (2026).

A infraestrutura ficou a cargo do integrante com maior experiência na área, que repassou ao restante do grupo tudo o que foi produzido, evitando que o conhecimento sobre o ambiente ficasse concentrado em uma só pessoa.

A organização das tarefas foi feita no GitHub Projects, onde as funcionalidades e seus prazos semanais foram registrados e acompanhados. O código foi versionado com Git, seguindo o modelo Git Flow. A branch main concentra as versões estáveis, e a branch develop reúne as funcionalidades já validadas. Cada nova funcionalidade era desenvolvida em uma branch própria, criada a partir da develop, e sua incorporação à develop era feita por meio de pull request, o que permitia a revisão do código pelos demais integrantes antes do merge. Dessa forma, só entrava na develop o que tivesse a qualidade da entrega garantida. A integração da develop com a main ocorria apenas quando o conjunto estava completo e funcionando.

As ferramentas de apoio ao desenvolvimento foram o Git e o GitHub, para versionamento, hospedagem do código e revisão por pull requests, o GitHub Projects, para gestão das tarefas, o Docker e o Docker Compose, para a execução padronizada dos serviços, e o WhatsApp e o Discord, para a comunicação. As demais tecnologias empregadas no sistema estão descritas na seção 3.

A construção do sistema seguiu a ordem natural das dependências entre seus componentes. Primeiro foi preparada a infraestrutura, com os serviços em contêineres. Em seguida, foram desenvolvidas a API e a fila de mensagens, depois

os workers e, por fim, o aplicativo móvel, que consome a API e recebe os alertas em tempo real. Essa ordem permitiu que cada etapa se apoiasse em uma base já funcional. O detalhamento do cronograma consta na seção 4.4.

4.2 Requisitos

Os requisitos do EcoPulse foram definidos a partir do enunciado da APS e das decisões de projeto tomadas pela equipe, que fixaram o escopo do protótipo e as regras de funcionamento de cada camada. Eles se dividem em requisitos funcionais (RF), que descrevem o que o sistema faz, e requisitos não funcionais (RNF), que descrevem como ele deve se comportar, em especial quanto à confiabilidade, ao desacoplamento e à consistência dos dados, foco da disciplina.

A prioridade segue dois níveis. Alta indica o que sustenta o núcleo da demonstração, isto é, o mapa com as camadas, o relato colaborativo e o alerta recebido em um segundo cliente. Média indica o que completa a experiência de uso. O Quadro 2 apresenta os requisitos.

Quadro 2 – Requisitos do EcoPulse

Código	Requisito	Prioridade
RF01	O sistema deve exibir um mapa mundial navegável (arrastar, aproximar e buscar endereço), iniciando na posição do GPS ou, na falta dela, em Ribeirão Preto.	Alta
RF02	O sistema deve exibir as camadas de calor e de qualidade do ar em células hexagonais H3, com dados obtidos da API Open-Meteo.	Alta
RF03	O sistema deve exibir as camadas de alagamento e de via obstruída como pontos no mapa, criados exclusivamente a partir dos relatos dos usuários.	Alta
RF04	O usuário deve poder ligar e desligar cada camada de forma independente e, ao tocar em uma célula ou ponto, consultar uma ficha com os dados do local.	Média

Código	Requisito	Prioridade
RF05	O usuário deve poder enviar um relato de alagamento ou de via obstruída, com gravidade (baixa, média ou alta), texto opcional, localização preenchida automaticamente pelo GPS e metadados registrados automaticamente (data e hora do envio e precisão da localização), sem login, sendo identificado apenas por um código gerado no aparelho.	Alta
RF06	O sistema deve validar cada relato (tipo, gravidade, coordenadas e limite de 30 relatos por dispositivo na última hora) e encaminhar os inválidos à fila de mensagens mortas.	Alta
RF07	O sistema deve agrupar os relatos por célula H3, marcar o incidente como confirmado quando houver dois relatos do mesmo tipo na mesma célula em até 2 horas e retirá-lo do mapa 6 horas após o último relato.	Alta
RF08	O sistema deve alertar o usuário, por meio de uma faixa no mapa com as ações de ver o local e dispensar o aviso por 30 minutos, quando ele estiver a até 500 metros de um incidente ativo de gravidade média ou alta, ou em uma célula com AQI acima de 150 ou temperatura acima de 35 °C.	Alta
RF09	O sistema deve entregar atualizações de camada e alertas aos clientes conectados em tempo real, por WebSocket, sem que o usuário atualize a tela.	Alta
RF10	O aplicativo deve guardar no aparelho os relatos feitos sem conexão e enviá-los em ordem quando a rede voltar, mantendo também a última camada consultada para visualização sem rede.	Média
RNF01	A API deve receber o relato, publicá-lo na fila e responder sem aguardar o processamento, que fica a cargo dos workers.	Alta
RNF02	Nenhum relato deve ser perdido em silêncio: filas duráveis, mensagens persistentes, confirmação (ack) somente após a gravação no PostgreSQL, até três novas tentativas com espera crescente (2, 4 e 8 segundos) e, esgotadas, envio à fila de mensagens mortas.	Alta
RNF03	O PostgreSQL deve ser o registro definitivo dos dados, e o Redis deve servir a leitura rápida com prazo de validade (10 minutos para clima e ar, 6 horas para incidentes), aceitando consistência eventual entre os dois.	Alta

Código	Requisito	Prioridade
RNF04	As consultas ao mapa devem trabalhar apenas a região visível, por retângulo e células H3, com limite de células por pedido, nunca o planeta inteiro.	Alta
RNF05	O sistema deve limitar as consultas à Open-Meteo por meio de cache, deixar a célula sem cor quando não houver dado (sem valores inventados) e exibir a atribuição das fontes.	Média
RNF06	A infraestrutura deve ser reproduzível com um único comando do Docker Compose, e o aplicativo deve funcionar no celular físico e no navegador.	Média
RNF07	O recebimento de relatos pela API deve continuar funcionando mesmo com os workers, o Redis ou a Open-Meteo indisponíveis: os relatos ficam na fila até serem processados.	Alta
RNF08	O alerta de proximidade deve chegar aos clientes conectados em até 5 segundos depois de o relato ser gravado, meta verificada nos testes (seção 4.5).	Média
RNF09	Todo alerta gerado deve ser gravado no PostgreSQL antes de ser publicado, e o cliente que se conectar ou reconectar deve receber os alertas recentes de sua região que ainda não tenha recebido.	Alta
RNF10	O processamento dos relatos deve poder ser dividido entre mais de uma instância de worker consumindo a mesma fila, de modo que a carga se distribua entre elas.	Média

Fonte: Elaborado pelos autores (2026).

4.3 Casos de uso

Os casos de uso descrevem como as pessoas e os sistemas externos interagem com o EcoPulse. O ator principal é o usuário do aplicativo, que explora o mapa, relata ocorrências e recebe alertas. A Open-Meteo participa como sistema externo, fornecendo temperatura e qualidade do ar ao worker de clima. Os demais componentes, como a API, a fila e os workers, fazem parte do sistema e aparecem nas descrições como etapas do processamento.

O Quadro 3 apresenta os cinco casos de uso do protótipo. Cada um traz o fluxo principal e, quando existe, o fluxo alternativo, e indica os requisitos do Quadro 2 que ele atende.

Quadro 3 – Casos de uso do EcoPulse

Código	Caso de uso	Descrição	Requisitos
UC01	Explorar o mapa e as camadas	O usuário abre o aplicativo, que centraliza o mapa no GPS (ou em Ribeirão Preto) e busca calor, qualidade do ar e incidentes da região visível. O usuário arrasta, aproxima, busca um endereço e liga ou desliga cada camada. *Alternativo:* se o clima ainda não chegou, o mapa continua navegável com o aviso “Atualizando camada”; se a Open-Meteo falhar, a célula fica sem cor ou mantém o dado anterior.	RF01 a RF04, RNF04, RNF05
UC02	Consultar a ficha de um local	Ao tocar em uma célula, o usuário vê temperatura, qualidade do ar, horário da leitura e a fonte dos dados. Ao tocar em um ponto, vê tipo, gravidade, situação (recebido ou confirmado) e há quanto tempo foi relatado.	RF04
UC03	Enviar relato	O usuário toca em “Relatar”, escolhe alagamento ou via obstruída, define a gravidade e envia, com a posição preenchida pelo GPS. A API publica o relato na fila e responde de imediato. Um worker o valida, grava no banco, confirma o incidente quando há dois relatos e gera o alerta. Os demais clientes veem o ponto por WebSocket. *Alternativo:* relato inválido ou acima de 30 por hora é recusado com mensagem; falhas passageiras geram novas tentativas e, esgotadas, a fila de mensagens mortas.	RF05 a RF07, RF09, RNF01 a RNF03, RNF07, RNF10

Código	Caso de uso	Descrição	Requisitos
UC04	Enviar relato sem conexão	Sem rede, o relato é guardado no aparelho como pendente e o usuário é avisado. Quando a conexão volta, os pendentes são enviados em ordem, e o usuário pode consultar a lista local (pendente, enviado ou falhou) e tentar de novo os que falharam.	RF10
UC05	Receber alerta de proximidade	Com o mapa aberto, o aplicativo informa a posição do usuário. Ao se aproximar de um risco conforme as regras do RF08, o sistema envia o alerta por WebSocket e o aplicativo exibe a faixa no topo, com as ações “Ver” e “Dispensar”. *Alternativo:* com o aplicativo fechado, o alerta não é entregue, pois a notificação push está fora desta versão.	RF08, RF09, RNF08, RNF09

Fonte: Elaborado pelos autores (2026).

4.4 Cronograma e etapas

O desenvolvimento foi organizado em seis ciclos semanais, entre outubro e novembro de 2026, seguindo a ordem de dependência entre os componentes descrita na seção 4.1. Cada ciclo correspondeu a uma sprint de uma semana e terminou com uma entrega integrada ao restante do sistema. O Quadro 4 apresenta as etapas, seus períodos e o critério usado para considerá-las concluídas.

Quadro 4 – Cronograma e etapas do EcoPulse

Sprint	Período	Etapa	Entregas	Critério de conclusão
1	12/10 a 18/10	Planejamento e infraestrutura	Levantamento de requisitos, definição das tecnologias, criação do repositório e do quadro no GitHub Projects, Docker Compose com PostgreSQL/PostGIS, Redis e RabbitMQ, e script SQL inicial	Todos os serviços sobem com um único comando e o painel do RabbitMQ abre
2	19/10 a 25/10	API e fila de mensagens	Contratos das mensagens, API com verificação de saúde, recebimento de relatos e leitura das camadas	Um relato enviado à API aparece na fila e a API responde sem aguardar o processamento
3	26/10 a 01/11	Workers de relatos e de clima	Consumo com confirmação manual, novas tentativas e fila de mensagens mortas, gravação no PostGIS, agrupamento por H3, consulta à Open-Meteo e cache no Redis	Relato inválido cai na fila de mensagens mortas, dois relatos próximos confirmam o incidente e a segunda consulta de uma região sai do cache
4	02/11 a 08/11	Tempo real e aplicativo	WebSocket com inscrição por região, aplicativo em React Native com mapa MapLibre, busca e camadas de calor, qualidade do ar, alagamento e via obstruída	O aplicativo abre no celular e no navegador e exibe as camadas da região
5	09/11 a 15/11	Relato, alerta e modo offline	Fluxo de relato com GPS, faixa de alerta de proximidade, fila local em SQLite e cache da última camada	Um relato feito em um cliente aparece no outro sem atualizar a tela, e o relato feito sem rede é enviado ao reconectar

Sprint	Período	Etapa	Entregas	Critério de conclusão
6	16/11 a 22/11	Testes integrados e fechamento	Testes ponta a ponta, ensaio da demonstração, relatório de código e revisão final da documentação	Roteiro da demonstração executado do início ao fim sem intervenção manual

Fonte: Elaborado pelos autores (2026).

4.5 Estratégia de testes

A estratégia de testes tem o objetivo de verificar se cada requisito do Quadro 2 foi atendido. Os testes são automatizados e executados por scripts contra o ambiente completo, iniciado com um único comando do Docker Compose, o mesmo usado na demonstração. Há testes de unidade, que verificam regras isoladas, como a distância do alerta; testes de integração, que verificam a API, a fila, o banco e o cache juntos; e testes ponta a ponta, que percorrem o fluxo completo entre dois clientes conectados à mesma API.

O Quadro 5 reúne os casos de teste e indica os requisitos que cada um cobre. As telas de mapa, camadas e busca (RF01 a RF04) dependem da interface e são verificadas por um roteiro visual no aparelho. Um caso é aprovado quando o resultado observado é igual ao esperado, e os resultados da execução são apresentados na seção 7.3.

Quadro 5 – Casos de teste do EcoPulse

Caso	Procedimento	Resultado esperado	Requisitos
CT01	Enviar um relato válido pelo cliente 1 e observar o cliente 2.	O relato aparece no cliente 2 sem atualizar a tela, e a API responde sem aguardar o worker.	RF05, RF09, RNF01
CT02	Enviar um relato com tipo, gravidade ou coordenadas inválidos.	O relato é recusado, e a mensagem vai para a fila de mensagens mortas.	RF06, RNF02
CT03	Enviar dois relatos do mesmo tipo na mesma célula H3 em menos de 2 horas.	O incidente passa a aparecer como confirmado.	RF07

Caso	Procedimento	Resultado esperado	Requisitos
CT04	Posicionar o usuário a até 500 metros de um incidente de gravidade média ou alta, ou em célula com AQI acima de 150 ou temperatura acima de 35 °C.	A faixa de alerta aparece. A ação de dispensar silencia o aviso por 30 minutos.	RF08
CT05	Consultar a mesma célula duas vezes seguidas e, depois, uma célula sem dado da Open-Meteo.	A segunda consulta usa o cache, sem nova chamada externa, e a célula sem dado fica sem cor.	RNF03, RNF05
CT06	Desligar a rede do aparelho, enviar três relatos e religar a rede.	Os relatos ficam pendentes no aparelho e são enviados em ordem.	RF10
CT07	Parar os workers, o Redis e o acesso à Open-Meteo e enviar relatos.	A API continua aceitando os relatos, que ficam na fila e são processados quando os serviços voltam.	RNF07
CT08	Desconectar o cliente 2, gerar um alerta e reconectar o cliente.	O cliente recebe o alerta que perdeu, lido do banco.	RNF09
CT09	Subir três workers, enviar vinte relatos e parar um dos workers durante o processamento.	Os relatos se dividem entre os workers, e os que continuam ativos concluem o trabalho do que parou.	RNF10
CT10	Medir o tempo entre a gravação do relato e a chegada do alerta ao cliente 2.	O alerta chega em até 5 segundos.	RNF08

Fonte: Elaborado pelos autores (2026).

A meta de tempo do alerta (RNF08) é de, no máximo, 5 segundos entre a gravação do relato no banco e a chegada do alerta ao segundo cliente. O caso CT10 mede esse intervalo pela diferença entre o horário de gravação e o horário de recebimento e é aprovado quando o resultado fica abaixo da meta.

4.6 Riscos e mitigação

O Quadro 6 reúne os principais riscos do projeto, o impacto de cada um, caso ocorra, e a ação planejada para evitá-lo ou reduzir seu efeito.

Quadro 6 – Riscos e mitigação do EcoPulse

Código	Risco	Impacto	Mitigação
R01	A Open-Meteo fica fora do ar ou lenta.	Média	Cache das leituras e célula sem cor quando não há dado, sem inventar valores (RNF03, RNF05).
R02	O limite de consultas da API pública é atingido.	Média	Consulta por célula H3 com cache, para não repetir chamadas externas (RNF05).
R03	Um worker ou o Redis deixa de funcionar.	Alta	Fila durável, mais de um worker e API que continua aceitando relatos (RNF02, RNF07, RNF10).
R04	Relatos falsos ou repetidos.	Média	Limite de 30 relatos por aparelho por hora e confirmação do incidente com dois relatos (RF06, RF07).
R05	GPS indisponível ou negado pelo usuário.	Média	O mapa abre em Ribeirão Preto quando não há localização (RF01).
R06	Conexão instável ou sem internet no celular.	Média	Os relatos ficam guardados no aparelho e são enviados em ordem quando a rede volta (RF10).
R07	Os alertas demoram mais que o esperado para chegar.	Alta	Mais de um worker na fila e tempo medido nos testes, com meta de 5 segundos (RNF08, RNF10).

Fonte: Elaborado pelos autores (2026).

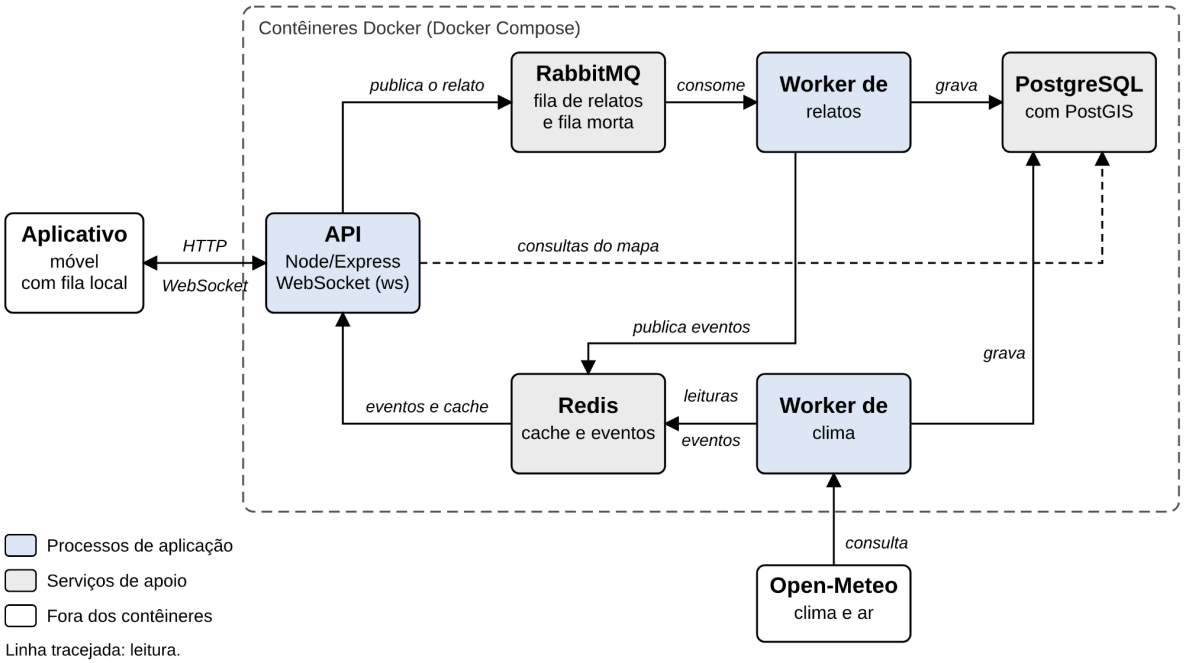
5 PROJETO E ARQUITETURA DO PROGRAMA

Esta seção apresenta o projeto do EcoPulse: a visão geral da arquitetura, os componentes e suas responsabilidades, o fluxo de eventos que leva um relato até o alerta, o modelo de dados e o comportamento do sistema diante de falhas. Os requisitos citados (RF e RNF) são os do Quadro 2.

5.1 Visão geral da arquitetura

O EcoPulse adota uma arquitetura orientada a eventos, em que o aplicativo móvel, a API, os workers, a fila de mensagens, o banco de dados e o cache cooperam sem se chamar diretamente. A Figura 1 mostra os componentes do sistema e o caminho dos dados entre eles.

Figura 1 – Arquitetura do EcoPulse



Fonte: Elaborado pelos autores (2026).

O aplicativo envia os relatos à API, que os coloca em uma fila do RabbitMQ. O worker de relatos retira cada relato da fila, grava-o no PostgreSQL e publica os eventos no Redis. A API, inscrita no Redis, repassa esses eventos por WebSocket aos aplicativos conectados. O worker de clima consulta a Open-Meteo e mantém o cache de calor e de qualidade do ar. Todos os serviços, exceto o aplicativo, rodam em contêineres Docker.

5.2 Microsserviços e responsabilidades

O Quadro 7 apresenta os componentes do sistema, a responsabilidade de cada um e os requisitos do Quadro 2 que ele atende. Os quatro processos de aplicação foram introduzidos na seção 3.2, e os demais são serviços de apoio.

Quadro 7 – Componentes e responsabilidades do EcoPulse

Componente	Responsabilidade	Requisitos
Aplicativo móvel	Exibe o mapa, as camadas e os alertas; envia relatos; guarda no aparelho os relatos sem rede e a última camada consultada.	RF01 a RF05, RF08, RF10
API	Recebe relatos e consultas, faz a validação mínima, publica o relato na fila e responde sem aguardar o processamento; envia por WebSocket as atualizações e os alertas aos clientes.	RF09, RNF01, RNF04
Worker de relatos	Consome a fila, valida o relato, grava no PostgreSQL, confirma o incidente e gera o alerta; publica os eventos no Redis.	RF06 a RF08, RNF02, RNF09, RNF10
Worker de clima	Consulta a Open-Meteo para as células visíveis, guarda as leituras e gera o alerta quando a célula passa de AQI 150 ou de 35 °C.	RF02, RF08, RNF05
RabbitMQ	Mantém a fila durável entre a API e os workers, com confirmação manual, novas tentativas e fila de mensagens mortas.	RNF01, RNF02, RNF07
PostgreSQL com PostGIS	Guarda os relatos, os incidentes, os alertas e as leituras como registro definitivo, com índice espacial.	RF03, RF07, RNF03
Redis	Guarda o cache de leituras e de incidentes ativos, com prazo de validade, e leva os eventos dos workers até a API.	RF09, RNF03

Fonte: Elaborado pelos autores (2026).

5.3 Fluxo de eventos e mensageria

O fluxo principal começa quando o usuário envia um relato. O aplicativo o envia à API, que faz uma validação mínima, publica a mensagem na fila durável do RabbitMQ e responde sem aguardar o processamento (RNF01).

Um worker de relatos retira a mensagem da fila e valida o relato quanto ao tipo, à gravidade, às coordenadas e ao limite de 30 relatos por aparelho na última

hora. Uma mensagem inválida vai direto para a fila de mensagens mortas. Sendo válida, o worker grava o relato no PostgreSQL, junto com a célula H3 do ponto, e só então confirma a mensagem ao RabbitMQ (RNF02).

Em seguida, o worker atualiza o incidente da célula. Com dois relatos do mesmo tipo em até 2 horas, o incidente passa a confirmado (RF07). Se a gravidade é média ou alta, o worker gera o alerta e o grava no PostgreSQL antes de publicá-lo (RNF09). Depois, publica no Redis os eventos de camada atualizada e de alerta gerado.

A API, inscrita nos canais do Redis, envia cada evento por WebSocket apenas aos clientes cujo retângulo do mapa ou cuja posição é afetada. É assim que o ponto aparece nos outros aparelhos sem atualizar a tela (RF09), e que o alerta chega aos usuários a até 500 m do incidente em até 5 segundos depois de o relato ser gravado (RF08, RNF08).

O worker de clima segue um caminho próprio. Ele consulta a Open-Meteo para as células visíveis nos mapas dos clientes, guarda cada leitura no cache por 10 minutos e publica o evento de camada atualizada. Se a leitura de uma célula passa de AQI 150 ou de 35 °C, o worker também gera o alerta dessa célula (RF08).

Quando uma falha passageira interrompe o processamento de um relato, a mensagem não é confirmada e volta para a fila, com até três novas tentativas, após 2, 4 e 8 segundos. Esgotadas as tentativas, ela segue para a fila de mensagens mortas (RNF02). Como a entrega do Redis aos assinantes não é garantida, o cliente que se conecta ou reconecta informa o horário do último alerta que recebeu, e a API envia os alertas recentes de sua região gravados depois desse horário (RNF09).

5.4 Modelo de dados

O PostgreSQL é o registro definitivo dos dados, e o Redis guarda cópias para leitura rápida, com prazo de validade (RNF03). O Quadro 8 resume o que cada um guarda e o que o aparelho mantém.

Quadro 8 – Dados do EcoPulse

Dado	Onde fica	Conteúdo principal
Relato	PostgreSQL	Tipo, gravidade, texto opcional, posição, célula H3, data e hora do envio, precisão da localização e código do aparelho.
Incidente	PostgreSQL e Redis (6 horas)	Tipo, gravidade, célula H3, número de relatos, situação (recebido ou confirmado) e horário do último relato.
Alerta	PostgreSQL	Origem (incidente, AQI ou temperatura), célula, posição, gravidade e horário de geração.
Leitura de clima e ar	PostgreSQL e Redis (10 minutos)	Célula H3, temperatura, AQI, horário da leitura e fonte (Open-Meteo).
Eventos	Redis	Eventos de camada atualizada e de alerta gerado, que a API repassa aos clientes.
Fila local	Aparelho (SQLite)	Relatos pendentes, enviados ou falhos e a última camada consultada (RF10).

Fonte: Elaborado pelos autores (2026).

Os relatos e os incidentes têm índice espacial no PostGIS, usado na busca pelo retângulo visível e pelo raio de 500 m (seção 3.4), e a célula H3 de cada um permite agrupá-los por região. O código gerado no aparelho identifica o autor do relato sem login e serve para aplicar o limite de 30 relatos por hora (RF05, RF06).

5.5 Tolerância a falhas e escalabilidade

O Quadro 9 mostra o que acontece quando uma parte do sistema falha e quais requisitos garantem esse comportamento.

Quadro 9 – Comportamento do EcoPulse diante de falhas

Falha	Comportamento do sistema	Requisitos
Worker parado	A API continua aceitando relatos, que ficam na fila até que um worker volte a processá-los.	RNF01, RNF07
Falha passageira no processamento	O relato volta à fila e é reprocessado até três vezes, com espera de 2, 4 e 8 segundos; depois, vai para a fila de mensagens mortas.	RNF02
Redis fora do ar	A leitura do mapa fica mais lenta, mas nenhum relato é perdido, pois o PostgreSQL é o registro definitivo.	RNF03, RNF07

Falha	Comportamento do sistema	Requisitos
Open-Meteo fora do ar	A API segue funcionando; a célula mantém a leitura em cache enquanto ela vale ou fica sem cor, sem valores inventados.	RNF05, RNF07
Cliente sem conexão	O aparelho guarda os relatos, envia-os em ordem quando a rede volta e recebe os alertas recentes que perdeu.	RF10, RNF09

Fonte: Elaborado pelos autores (2026).

Quanto à escalabilidade, o processamento dos relatos pode ser dividido entre mais de uma instância do worker consumindo a mesma fila, de modo que a carga se distribui entre elas (RNF10). Como cada consulta e cada incidente envolvem poucas células H3, os dados e a carga também podem ser divididos por região, que é a ideia de particionamento geoespacial da disciplina.

No protótipo, o PostgreSQL e o Redis rodam, cada um, em um único contêiner, sem divisão do banco em partes (sharding) nem Redis Cluster. Essa é uma limitação assumida desta versão, assim como a notificação push com o aplicativo fechado. A evolução prevista é distribuir o banco e o cache por faixas de células H3 e acrescentar o push.

6 RELATÓRIO DE CÓDIGO

[Máximo de 10 páginas.]

6.1 Estrutura do repositório

6.2 Trechos relevantes do código

7 APRESENTAÇÃO EM FUNCIONAMENTO

7.1 Ambiente de execução

7.2 Demonstração das funcionalidades

7.3 Resultados dos testes

8 CONSIDERAÇÕES FINAIS

[Retome os objetivos da seção 1 e mostre como foram atendidos; limitações e trabalhos futuros.]

REFERÊNCIAS

BRODSKY, Isaac. **H3: Uber's hexagonal hierarchical spatial index**. [S. l.]: Uber Engineering, 27 jun. 2018. Disponível em: https://www.uber.com/br/en/blog/h3/. Acesso em: 17 out. 2026.

CEMADEN. **3. Hidrologia**. Brasília, DF: Ministério da Ciência, Tecnologia e Inovação, 2022. Disponível em: https://www.gov.br/cemaden/pt-br/assuntos/pesquisa/3-hidrologia. Acesso em: 17 out. 2026.

CONFESSOR, Arthur Octávio Oliveira et al. Proposta de aplicação móvel para coleta de dados de pesquisa em áreas com ou sem internet. In: ESCOLA REGIONAL DE INFORMÁTICA DE MATO GROSSO, 14., 2025, Pontes e Lacerda. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2025. p. 418-422. DOI: 10.5753/eri-mt.2025.16963.

DANIEL, João F. L. et al. Byron, an event-driven microservices framework. In: ESCOLA REGIONAL DE ALTO DESEMPENHO DE SÃO PAULO, 11., 2020, on-line. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2020. p. 66-69. DOI: 10.5753/eradsp.2020.16888.

DODMAN, D. et al. Cities, settlements and key infrastructure. In: PÖRTNER, H.-O. et al. (ed.). **Climate change 2022**: impacts, adaptation and vulnerability. Contribution of Working Group II to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change. Cambridge: Cambridge University Press, 2022. p. 907-1040. DOI: 10.1017/9781009325844.008.

ESCOBAR-SILVA, E. V. et al. Water-related disasters in Brazil: an assessment from 1991 to 2024. **Environmental Research Letters**, Bristol, v. 21, art. 074015, 10 abr. 2026. DOI: 10.1088/1748-9326/ae5991.

FERREIRA, Saulo; ANDRADE, Ermeson; MENDONÇA, Júlio. Uma abordagem experimental para avaliar os níveis de consistência do banco de dados NoSQL Cassandra. In: SIMPÓSIO EM SISTEMAS COMPUTACIONAIS DE ALTO DESEMPENHO, 22., 2021, Belo Horizonte. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2021. p. 156-167. DOI: 10.5753/wscad.2021.18520.

FETTE, I.; MELNIKOV, A. **The WebSocket protocol**. [S. l.]: Internet Engineering Task Force, dez. 2011. (RFC, 6455). DOI: 10.17487/RFC6455.

GOODCHILD, Michael F. Citizens as voluntary sensors: spatial data infrastructure in the world of Web 2.0. **International Journal of Spatial Data Infrastructures Research**, [S. l.], v. 2, p. 24-32, 2007.

KLEPPMANN, Martin. **Designing data-intensive applications**: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

OLIVEIRA, Lucas Bicalho; FAZENDA, Álvaro Luiz. Avaliação comparativa de desempenho entre serviços de mensageria em ambiente de computação em nuvem. In: ESCOLA REGIONAL DE ALTO DESEMPENHO DE SÃO PAULO, 17., 2026, São

Paulo. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2026. p. 5-8. DOI: 10.5753/eradsp.2026.30729.

OPEN-METEO. **Open-Meteo**: free weather API. [S. l.], [s. d.]. Disponível em: https://github.com/open-meteo/open-meteo. Acesso em: 17 out. 2026.

POSTGIS DEVELOPMENT GROUP. **PostGIS 3.6.5dev manual**. [S. l.]: PostGIS Development Group, 2026. Disponível em: https://postgis.net/docs/manual-3.6/br/index.html. Acesso em: 17 out. 2026.

RABBITMQ. **Consumer acknowledgements and publisher confirms**. [S. l.]: Broadcom, [s. d.]. Disponível em: https://www.rabbitmq.com/docs/confirms. Acesso em: 17 out. 2026.

REDIS. **Redis Pub/Sub**. [S. l.]: Redis, [s. d.]. Disponível em: https://redis.io/docs/latest/develop/pubsub/. Acesso em: 17 out. 2026.

SANTOS, João Luís dos et al. Otimização de um banco de dados geográficos utilizando PostGIS. In: SIMPÓSIO BRASILEIRO DE SENSORIAMENTO REMOTO, 18., 2017, Santos. **Anais [...]**. Santos: INPE, 2017. p. 2224-2231.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Distributed systems**: principles and paradigms. 2. ed. Upper Saddle River: Pearson Prentice Hall, 2007.

TENÓRIO, Ricardo Bruno de Araújo. **Uma proposta para uso de cache distribuído no contexto do TCE-RN**. 2022. 19 f. Trabalho de Conclusão de Curso (Residência em Tecnologia da Informação) – Instituto Metrópole Digital, Universidade Federal do Rio Grande do Norte, Natal, 2022. Disponível em: https://repositorio.ufrn.br/handle/123456789/48324. Acesso em: 17 out. 2026.

TRINDADE, Leon Valentim Porto. **Análise do desempenho da virtualização leve para ambientes com edge computing baseada em NFV**. 2018. Dissertação (Mestrado em Engenharia Elétrica) – Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2018. Disponível em: https://www.gta.ufrj.br/ftp/gta/TechReports/Leon18.pdf. Acesso em: 17 out. 2026.

ANEXO A – Ficha de APS

[Inserir a Ficha de APS fornecida pela UNIP. Se for um documento elaborado por você, troque para APÊNDICE.]